MediRAG

Multi-Agent AI Medical Report Analyzer

Project Documentation • Data Science Submission

Author: Uppala Venkata Satya Srinivas

1. Project Overview

MediRAG is an AI-powered system designed to analyze medical reports, prescriptions, and lab test results using Document AI, a Multi-Agent pipeline, Retrieval-Augmented Generation (RAG), and Large Language Models (LLMs). The system automatically extracts medical data, evaluates laboratory values, and generates simple, human-readable health explanations to help patients understand their medical documents.

2. Key Features

- Document AI Processing – Supports PDF reports and scanned image documents
- OCR Integration – Extracts text from scanned reports using Tesseract OCR
- Multi-Agent Workflow – Modular AI agents each handle a distinct analysis task
- Medical Entity Extraction – Detects lab test values and medical parameters
- Patient Information Extraction – Identifies patient details from reports
- Automated Medical Analysis – Compares results with standard normal ranges
- AI-Generated Explanations – Provides simplified health insights via LLMs
- Vector Database Storage – Stores processed reports in ChromaDB for future retrieval
- Report ID System – Unique IDs allow users to retrieve previously analyzed reports

3. Technology Stack

Category	Technology	Purpose
Backend	Python, FastAPI, Streamlit	Server & UI framework
Multi-Agent Framework	LangGraph	Agent orchestration
LLM Provider	Groq API (Llama 3 / Mixtral)	AI text generation
Document Processing	PyPDF, Tesseract OCR	PDF & image reading
Embeddings	Sentence Transformers	Text vectorization
Vector Database	ChromaDB	Report storage & retrieval

Frontend	HTML, CSS, JavaScript	User interface

4. System Architecture

MediRAG uses a multi-agent pipeline built with LangGraph. Each report passes through the following sequential agents:

- Step 1 → User uploads a Medical Report (PDF or scanned image)
- Step 2 → Document Reader processes the file (PDF Parser / OCR via Tesseract)
- Step 3 → Agent 1: File Classifier — identifies document type
- Step 4 → Agent 2: Medical Entity Extractor — extracts lab values & parameters
- Step 5 → Agent 3: Patient Information Agent — identifies patient details
- Step 6 → Agent 4: Medical Analysis Agent — compares values with normal ranges
- Step 7 → Agent 5: Response Generator — creates human-readable explanation
- Step 8 → Vector Database Storage — report saved with a unique Report ID
- Step 9 → User receives Medical Explanation + Report ID for future retrieval

5. Project Structure

```
medirag-ai/
├── agents/                 # AI agents for classification, extraction &
analysis
│   ├── classifier_agent.py
│   ├── entity_extractor_agent.py
│   ├── patient_info_agent.py
│   ├── analysis_agent.py
│   └── response_agent.py
├── rag/                    # Retrieval-Augmented Generation components
│   ├── embedding_model.py
│   ├── vector_store.py
│   └── retrieval_agent.py
├── utils/                  # Helper utilities
│   ├── pdf_reader.py
│   ├── ocr_reader.py
│   └── report_id_generator.py
├── workflows/              # LangGraph multi-agent workflow
│   └── medical_workflow.py
├── frontend/               # Web UI (HTML, CSS, JS)
├── chroma_db/              # Vector database storage
├── app.py                  # Main application entry point
├── flask_app.py            # Flask API server
├── streamlit_app.py        # Streamlit UI
├── config.py               # Configuration & settings
├── state.py                # Shared agent state
```

```
├── llm_model.py            # LLM model loader
├── requirements.txt        # Python dependencies
└── .env                    # API keys (not committed to git)
```

6. Installation & Setup

Step 1 – Clone the Repository

```
git clone https://github.com/yourusername/medirag-ai.git
cd medirag-ai
```

Step 2 – Create a Virtual Environment

```
python -m venv venv
venv\Scripts\activate          # Windows
source venv/bin/activate       # Mac / Linux
```

Step 3 – Install Dependencies

```
pip install -r requirements.txt
```

Step 4 – Add API Key

Create a .env file in the project root and add your Groq API key:

```
GROQ_API_KEY=your_groq_api_key_here
```

Step 5 – Run the Application

```
streamlit run app.py
```

Open your browser and navigate to: http://localhost:8501

7. Example Output

Extracted Medical Values (JSON)

```
{ "hemoglobin": 10.5, "blood_sugar": 113, "platelets": 205000, "cholesterol":
194 }
```

Medical Analysis Result

- Hemoglobin → Low (Possible anemia)
- Blood Sugar → Slightly elevated
- Cholesterol → Near upper limit

Report ID

```
Report ID: MED-a73b9f21
```

Users can enter this ID at any time to retrieve and re-query their previously analyzed report.

8. RAG-Based Retrieval

MediRAG supports Retrieval-Augmented Generation (RAG) to answer natural language questions about a stored report. Example:

```
Query: What does my hemoglobin level indicate?
```

The AI retrieves the stored report from ChromaDB and generates a contextual explanation using the LLM.

9. Future Improvements

- Prescription understanding agent
- Chat interface for interactive medical report Q&A
- Integration with medical knowledge bases (WHO / clinical guidelines)
- Cloud deployment for public access
- Doctor recommendation system

10. Medical Disclaimer

This project is intended only for educational and informational purposes. It does not replace professional medical advice. Users should always consult a qualified healthcare professional for diagnosis and treatment.

Author

Uppala Venkata Satya Srinivas
Data Science & AI Enthusiast
Portfolio: https://uppalasrinu.vercel.app/
LinkedIn: https://www.linkedin.com/in/srinuuppala/